Assignment 2

Matthew Boubin | MJB9353 | Cloud Computing | Prof. Sambit Sahu

Part 1: Creating an Application

I downloaded an archive of the project, and unpackaged it into a directory on my machine locally. I created a github repository to track changes to the project: https://github.com/boubinmj/cc-web-system

The application is the provided Flask To-Do app backed by MongoDB, kept in `~/Documents/cc-web-system`.

- `app.py`: Flask routes to list, add, update, complete, search and remove tasks
- `templates/`: the Jinja HTML pages
- `requirements.txt`: `flask`, `pymongo`

The app finds MongoDB through two environment variables, so the same image runs under Docker Compose and Kubernetes without code changes:

```python
mongodb_host = os.environ.get('MONGO_HOST', 'localhost')

mongodb_port = int(os.environ.get('MONGO_PORT', '27017'))

client = MongoClient(mongodb_host, mongodb_port)
```

Part 2: Containerizing the Application on Docker

The app runs as two containers: one built from the Dockerfile for Flask, and the official `mongo:7` image with a named volume for the data.

Flask Dockerfile

```
Shell
FROM python:3.12-slim

WORKDIR /app

# Install dependencies first so this layer is cached between code changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

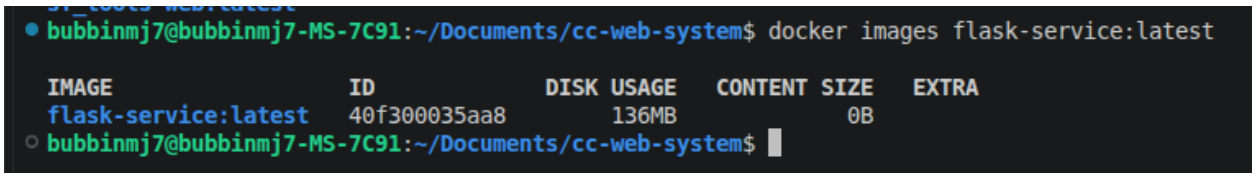
# Copy the app (app.py, templates/, static/)
COPY . .

EXPOSE 5000

# Must bind to 0.0.0.0, or the app won't be reachable from outside the container
CMD ["flask", "--app", "app", "run", "--host=0.0.0.0", "--port=5000"]
```

Building the image:

```
Shell
$ docker build -t mjb9353/todo-app:v1 .
$ docker images mjb9353/todo-app:v1
```



Docker Compose:

```
Shell

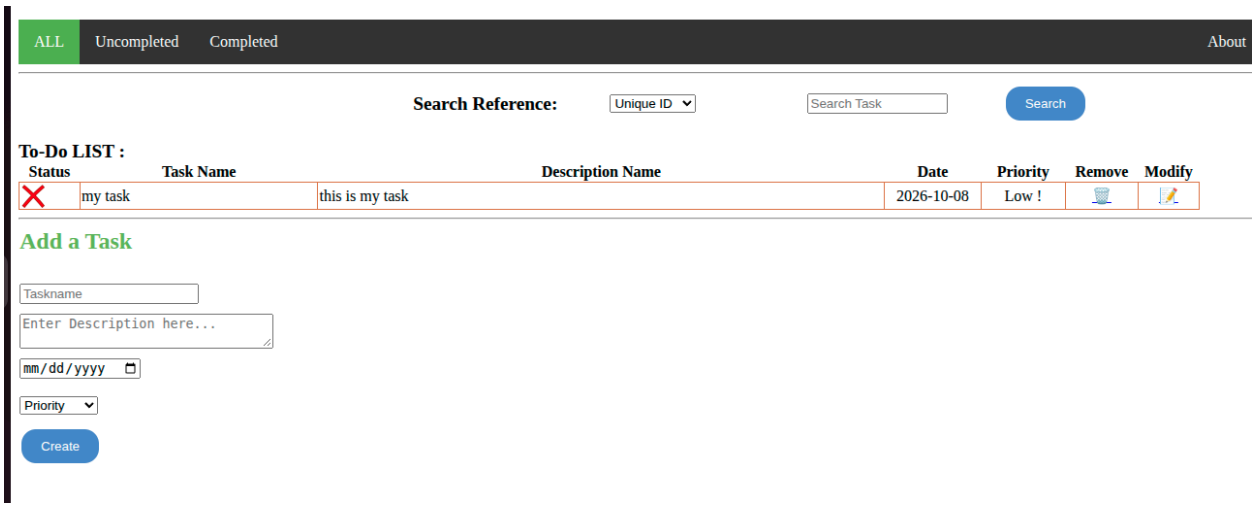
services:
  flask-app:
    build: .
    image: mjb9353/todo-app:v1
    container_name: flask-app
```

```
    ports:
      - "5000:5000"
    environment:
      MONGO_HOST: mongodb
      MONGO_PORT: "27017"
    depends_on:
      - mongodb

  mongodb:
    image: mongo:7
    container_name: mongodb
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db

volumes:
  mongo-data:
```

Build and run locally




```shell
docker build -t mjb9353/todo-app:v1 .
docker push mjb9353/todo-app:v1
```

Part 3: Deploying the Application on Minikube

Start Minikube using the command-line interface.

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ minikube start --driver=docker
😄  minikube v1.39.0 on Ubuntu 24.04
✨  Using the docker driver based on user configuration
📌  Using Docker driver with root privileges
❗  For an improved experience it's recommended to use Docker Engine instead of Docker Desktop.
Docker Engine installation instructions: https://docs.docker.com/engine/install/#server
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.51 ...
💾  Downloading Kubernetes v1.37.0 preload ...
    > preloaded-images-k8s-v18-v1...:  347.58 MiB / 347.58 MiB  100.00% 72.73 M
    > gcr.io/k8s-minikube/kicbase:  507.60 MiB / 507.61 MiB  100.00% 58.48 MiB
🔥  Creating docker container (CPUs=2, Memory=7702MB) ...
📦  Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
🔗  Configuring CNI (Container Networking Interface) ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass

❗  /snap/bin/kubectl is version 1.35.9, which may have incompatibilities with Kubernetes 1.37.0.
    ▪ Want kubectl v1.37.0? Try 'minikube kubectl -- get pods -A'
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

```
🏄  Done! kubectl is now configured to use "minikube" cluster and "defaul
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get nodes -o wide
NAME       STATUS   ROLES           AGE     VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE                         KERNEL-VERSION           CONTAINER-RUNTIME
minikube   Ready    control-plane   3m39s   v1.37.0   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm)   7.0.14-linuxkit (amd64)   containerd://2.3.4
```

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:42651
CoreDNS is running at https://127.0.0.1:42651/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```

Manifests

Flask

Shell

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: flask-app
spec:
 replicas: 2
 selector:
   matchLabels:
     app: flask-app
 template:
   metadata:
     labels:
       app: flask-app
   spec:
     containers:
      - name: flask-app
        image: mjb9353/todo-app:v1
        ports:
          - containerPort: 5000
        env:
          - name: MONGO_HOST
            value: "mongo"
          - name: MONGO_PORT
            value: "27017"
---
apiVersion: v1
kind: Service
metadata:
 name: flask-service
spec:
 type: NodePort
 selector:
   app: flask-app
 ports:
  - port: 80
    targetPort: 5000
```

Mongo

```XML
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: mongo-pvc
spec:
 accessModes: [ReadWriteOnce]
 resources:
   requests:
     storage: 1Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
 name: mongo
spec:
 replicas: 1
 selector:
   matchLabels:
     app: mongo
 template:
   metadata:
     labels:
       app: mongo
   spec:
     containers:
      - name: mongo
        image: mongo:7
        ports:
          - containerPort: 27017
        volumeMounts:
          - name: mongo-data
            mountPath: /data/db
     volumes:
      - name: mongo-data
        persistentVolumeClaim:
          claimName: mongo-pvc
---
apiVersion: v1
kind: Service
metadata:
 name: mongo
spec:
```

```
selector:
  app: mongo
ports:
 - port: 27017
   targetPort: 27017
```

Apply both of these manifests to the cluster:

```
XML
kubectl apply -f k8s/
```

Minikube Deployment

```shell
kubectl get deployments,pods,svc,pvc
```

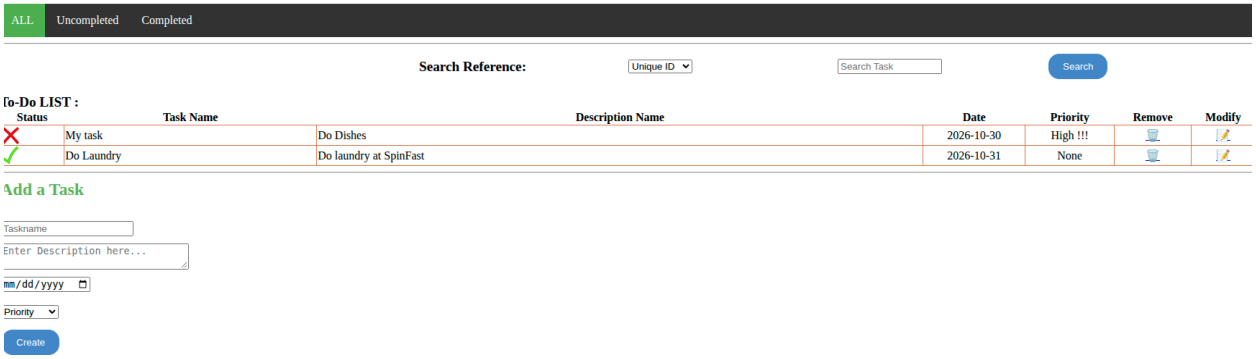
```
^Cbubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get deployments,pods,svc,pvc
NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/flask-app     2/2     2            2           2m7s
deployment.apps/mongo         1/1     1            1           2m7s

NAME                              READY   STATUS    RESTARTS   AGE
pod/flask-app-648b5fc64d-9lx87    1/1     Running   0          2m7s
pod/flask-app-648b5fc64d-l9gjh    1/1     Running   0          2m7s
pod/mongo-786b8fc5b9-h5r9z        1/1     Running   0          2m7s

NAME                    TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
service/flask-service   NodePort    10.104.138.113   <none>        80:31237/TCP   2m7s
service/kubernetes      ClusterIP   10.96.0.1        <none>        443/TCP        77m
service/mongo           ClusterIP   10.99.228.21     <none>        27017/TCP      2m7s

NAME                                  STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/mongo-pvc       Bound    pvc-fb68bf5d-6f0f-4e02-975f-83163cd3f5f0   1Gi        RWO            standard       <unset>                 2m7s
```

App running on Minikube in the browser:

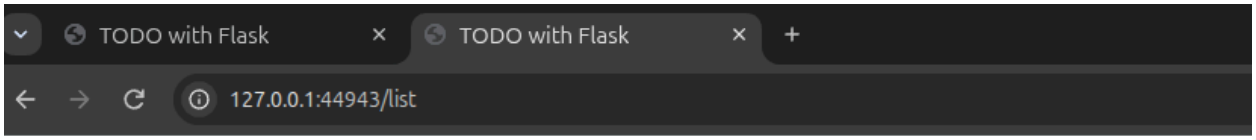


Delete Pod:

```shell
kubectl delete pod mongo-786b8fc5b9-t77pf
```

Mongo pod recovers:

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl delete pod mongo-786b8fc5b9-t77pf
pod "mongo-786b8fc5b9-t77pf" deleted from default namespace
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods -w
NAME                          READY   STATUS    RESTARTS   AGE
flask-app-648b5fc64d-9lx87    1/1     Running   0          9m20s
flask-app-648b5fc64d-l9gjh    1/1     Running   0          9m20s
mongo-786b8fc5b9-hvsc8        1/1     Running   0          5s
```



ToDo Reminder

ALL | Uncompleted | Completed

Se

To-Do LIST :

Status	Task Name	
✗	My task	Do Dishes
✓	Do Laundry	Do laundry at SpinFast

Add a Task

Taskname

Enter Description here...

mm/dd/yyyy

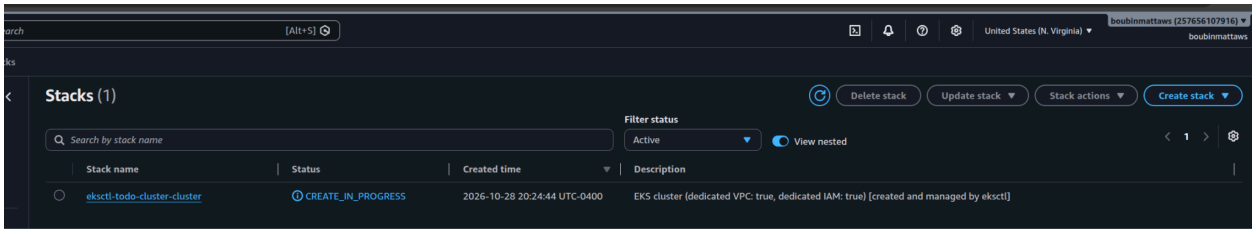
Priority

Create

Part 4: EKS Deployment

The same app runs on a 2-node AWS EKS cluster in us-east-1. Two things differ from Minikube: the Flask Service is a `LoadBalancer`, which provisions a Load Balancer, and MongoDB's volume is an EBS disk created through the EBS CSI driver.

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ eksctl create cluster \
  --name todo-cluster \
  --region us-east-1 \
  --nodegroup-name todo-nodes \
  --node-type t3.medium \
  --nodes 2 \
  --with-oidc \
  --managed
2026-10-28 20:24:44 [ℹ]  eksctl version 0.231.0
2026-10-28 20:24:44 [ℹ]  using region us-east-1
```



eksctl writes the cluster's credentials to `~/.kube/config` and switches kubectl to it.

```
odes'
2026-10-28 20:40:36 [✔]  EKS cluster "todo-cluster" in "us-east-1" region is ready
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl config current-context
matt-dev@todo-cluster.us-east-1.eksctl.io
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get nodes
NAME                             STATUS   ROLES    AGE     VERSION
ip-192-168-28-98.ec2.internal    Ready    <none>   7m9s    v1.34.12-eks-577a51e
ip-192-168-57-211.ec2.internal   Ready    <none>   7m11s   v1.34.12-eks-577a51e
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$
```

Install EBS CSI Drivers for Mongo Volume

Shell

```
eksctl create iamserviceaccount
 --name ebs-csi-controller-sa --namespace kube-system
 --cluster todo-cluster --region us-east-1
 --role-name AmazonEKS_EBS_CSI_DriverRole --role-only
 --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy
--approve
```

```
eksctl create addon --name aws-ebs-csi-driver
  --cluster todo-cluster --region us-east-1
  --service-account-role-arn arn:aws:iam::$(aws sts get-caller-identity
  --query Account --output text):role/AmazonEKS_EBS_CSI_DriverRole
  --force
```

The EKS manifests in `eks/` are copies of the Minikube ones with two changes: the Mongo PVC sets `storageClassName: gp3`, and the Flask Service uses `type: LoadBalancer`. A StorageClass tells the EBS CSI driver to create gp3 EBS volumes:

XML

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
parameters:
  type: gp3
```

Deploy:

Shell

```
kubectl apply -f eks/
```

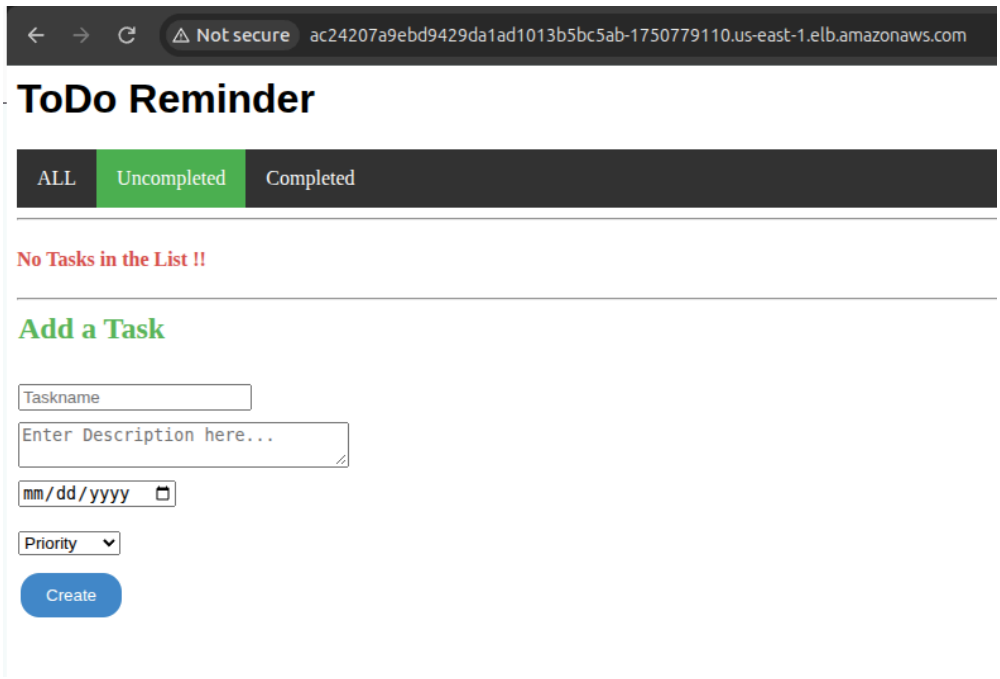
Once everything is running kubectl shows pods, services, and volume

```
^Cbubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods,svc,pvc
NAME                                READY   STATUS    RESTARTS   AGE
pod/flask-app-685dff69d-mdg8d       1/1     Running   0          3m17s
pod/flask-app-685dff69d-thn54       1/1     Running   0          3m17s
pod/mongo-bd8ddb5f4-9mmm8           1/1     Running   0          3m17s

NAME                    TYPE           CLUSTER-IP       EXTERNAL-IP
                   PORT(S)        AGE
service/flask-service   LoadBalancer   10.100.181.35    ac24207a9ebd9429da1ad1013b5bc5ab-1750779110.us-east-
1.elb.amazonaws.com   80:30773/TCP   3m17s
service/kubernetes      ClusterIP      10.100.0.1       <none>
                   443/TCP        21m
service/mongo           ClusterIP      10.100.191.114   <none>
                   27017/TCP      3m17s

NAME                              STATUS   VOLUME                                     CAPACITY   ACCESS MODE
S   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/mongo-pvc   Bound    pvc-68b5f4d3-3dbb-4679-a234-b1a944d33367   1Gi        RWO
    gp3            <unset>                 3m17s
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$
```

The service is running on the EXTERNAL-IP:
ac24207a9ebd9429da1ad1013b5bc5ab-1750779110.us-east-1.elb.amazonaws.com



Part 5: Deployments and ReplicaSets

Replicas

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods,svc
NAME                             READY   STATUS    RESTARTS   AGE
pod/flask-app-685dff69d-mdg8d    1/1     Running   0          21h
pod/flask-app-685dff69d-thn54    1/1     Running   0          21h
pod/mongo-bd8ddb5f4-9mmm8        1/1     Running   0          21h

NAME                    TYPE           CLUSTER-IP       EXTERNAL-IP                                                               PORT(S)        AGE
service/flask-service   LoadBalancer   10.100.181.35    ac24207a9ebd9429da1ad1013b5bc5ab-1750779110.us-east-1.elb.amazonaws.com   80:30773/TCP   21h
service/kubernetes      ClusterIP      10.100.0.1       <none>                                                                    443/TCP        22h
service/mongo           ClusterIP      10.100.191.114   <none>                                                                    27017/TCP      21h
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ grep -n "replicas" eks/flask.yaml
6:  replicas: 2
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$
```

Apply replica set for flask application

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl apply -f eks/flask.yaml
deployment.apps/flask-app unchanged
service/flask-service unchanged
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get deployment flask-app
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
flask-app   2/2     2            2           21h
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get rs
NAME                   DESIRED   CURRENT   READY   AGE
flask-app-685dff69d    2         2         2       21h
mongo-bd8ddb5f4        1         1         1       21h
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$
```

Now we will delete a pod and watch it get replaced:

```
^Cbubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods -w
NAME                           READY   STATUS    RESTARTS   AGE
flask-app-685dff69d-mdg8d      1/1     Running   0          21h
flask-app-685dff69d-thn54      1/1     Running   0          21h
mongo-bd8ddb5f4-9mmm8          1/1     Running   0          21h
```

Deleted first pod flask-app (`mdg8d`):

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods
NAME                           READY   STATUS    RESTARTS   AGE
flask-app-685dff69d-mdg8d      1/1     Running   0          21h
flask-app-685dff69d-thn54      1/1     Running   0          21h
mongo-bd8ddb5f4-9mmm8          1/1     Running   0          21h
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl delete pod flask-app-685dff69d-mdg8d
pod "flask-app-685dff69d-mdg8d" deleted from default namespace
```

Pod recovery:

Kubernetes deletes the pod, and a new pod (`d6bgg`) is stood up in its place

```
^Cbubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods -w
NAME                           READY   STATUS    RESTARTS   AGE
flask-app-685dff69d-mdg8d      1/1     Running   0          21h
flask-app-685dff69d-thn54      1/1     Running   0          21h
mongo-bd8ddb5f4-9mmm8          1/1     Running   0          21h
flask-app-685dff69d-mdg8d      1/1     Terminating   0          21h
flask-app-685dff69d-mdg8d      1/1     Terminating   0          21h
flask-app-685dff69d-d6bgg      0/1     Pending       0          0s
flask-app-685dff69d-d6bgg      0/1     Pending       0          1s
flask-app-685dff69d-d6bgg      0/1     ContainerCreating   0          1s
flask-app-685dff69d-d6bgg      0/1     ContainerCreating   0          1s
flask-app-685dff69d-d6bgg      1/1     Running             0          2s
flask-app-685dff69d-mdg8d      0/1     Error               0          21h
flask-app-685dff69d-mdg8d      0/1     Error               0          21h
flask-app-685dff69d-mdg8d      0/1     Error               0          21h
```

Scale to increase to 4 replicas:

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods
NAME                           READY   STATUS    RESTARTS   AGE
flask-app-685dff69d-d6bgg      1/1     Running   0          3m35s
flask-app-685dff69d-thn54      1/1     Running   0          22h
mongo-bd8ddb5f4-9mmm8          1/1     Running   0          22h
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl scale deployment flask-app --replicas=4
deployment.apps/flask-app scaled
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get rs
NAME                     DESIRED   CURRENT   READY   AGE
flask-app-685dff69d      4         4         4       22h
mongo-bd8ddb5f4          1         1         1       22h
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods
NAME                           READY   STATUS    RESTARTS   AGE
flask-app-685dff69d-5jw2r      1/1     Running   0          19s
flask-app-685dff69d-bs9lp      1/1     Running   0          19s
flask-app-685dff69d-d6bgg      1/1     Running   0          5m54s
flask-app-685dff69d-thn54      1/1     Running   0          22h
mongo-bd8ddb5f4-9mmm8          1/1     Running   0          22h
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$
```

Scaled back down to 2 (back in sync with the YAML file):

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl scale deployment flask-app --replicas=2
deployment.apps/flask-app scaled
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get rs
NAME                   DESIRED   CURRENT   READY   AGE
flask-app-685dff69d    2         2         2       22h
mongo-bd8ddb5f4        1         1         1       22h
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$
```

Part 6: Rolling Updates:

Local docker build of v2 that we will use for the rollout.

Added timeout for mongo client

```python
client = MongoClient(mongodb_host, mongodb_port, serverSelectionTimeoutMS=2000)
```

Added health/ and ready/ routes (for part 7):

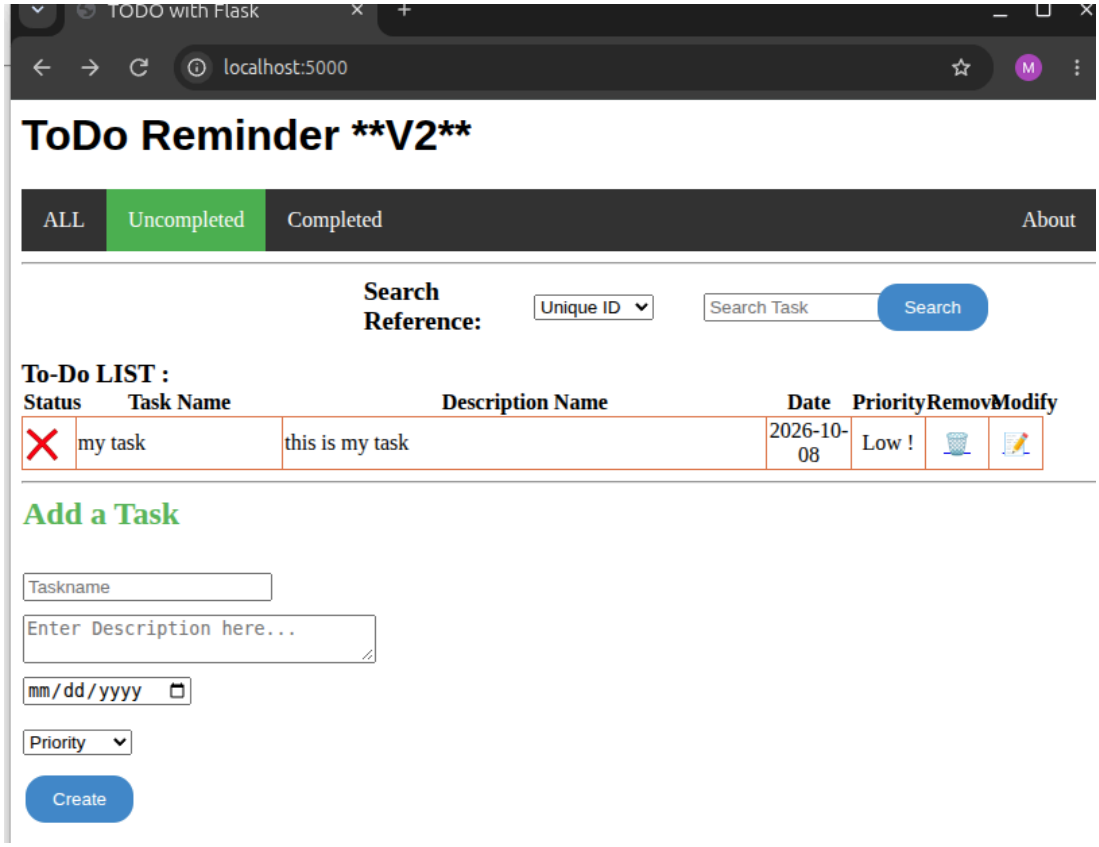
```python
#### HEALTH CHECK ROUTES #####
HEALTHY = True # Health Flag
@app.route("/health")
def health():
   # Checks flask service
   if HEALTHY:
       return {"status": "ok"}, 200
   return {"status": "unhealthy"}, 500

@app.route("/ready")
def ready():
   # depends on MongoDB being available
   try:
       client.admin.command("ping")
       return {"status": "ready"}, 200
   except Exception:
       return {"status": "mongo unavailable"}, 503

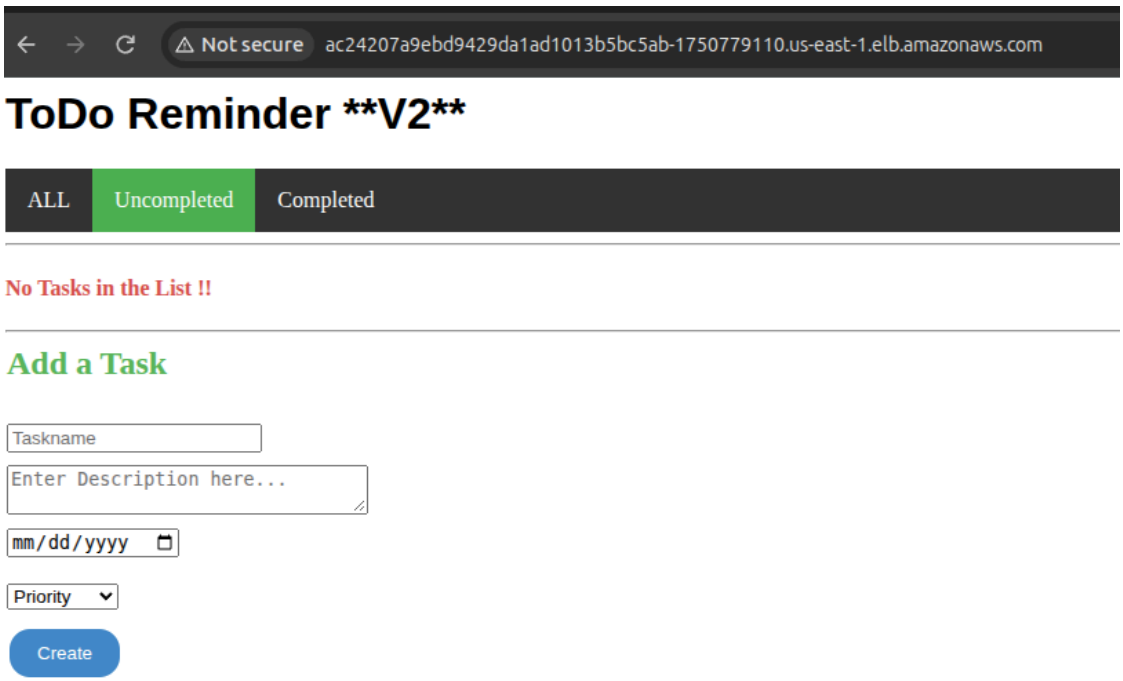
@app.route("/fail")
def fail():
   # Deliberately break liveness, used to test the probe in Part 7
   global HEALTHY
   HEALTHY = False
   return {"status": "liveness will now fail"}, 200

#### HEALTH CHECK ROUTES #####
```

Local rollout



Live rollout



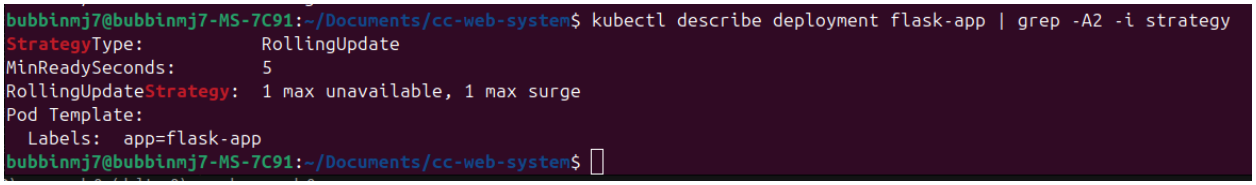
Push the new image to docker hub:

```
Shell
docker build -t mjb9353/todo-app:v2 .
docker push mjb9353/todo-app:v2
```

Now spec is added to flask.yaml

```
XML
spec:
 replicas: 2
 minReadySeconds: 5
 strategy:
   type: RollingUpdate
   rollingUpdate:
     maxUnavailable: 1
     maxSurge: 1
```

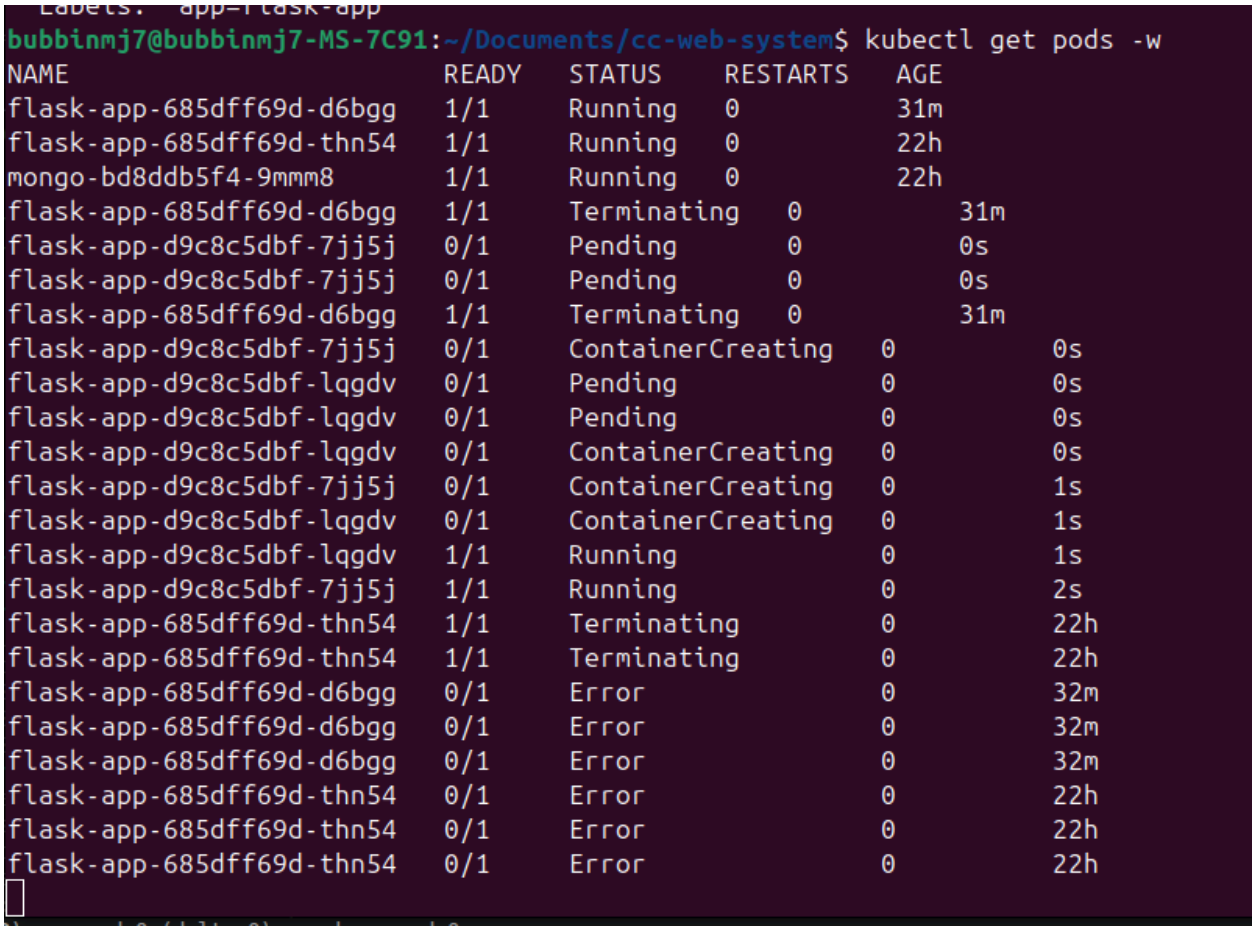
Apply Strategy:

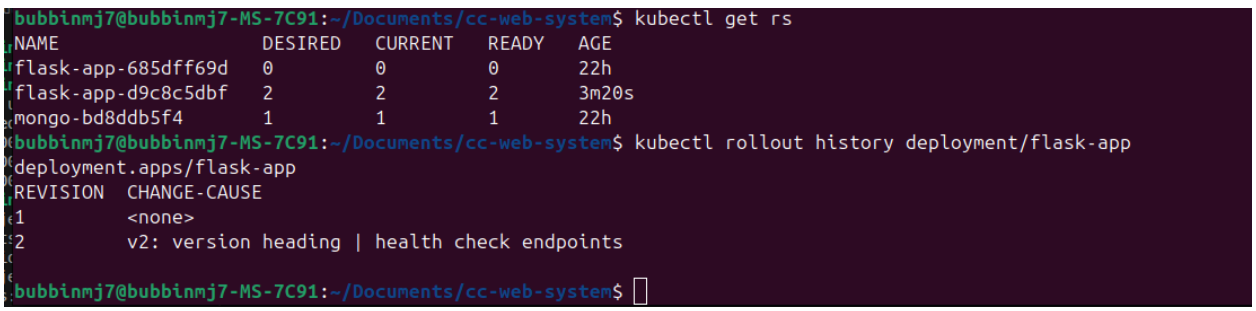


Trigger rollout:

```
XML
kubectl set image deployment/flask-app flask-app=mjb9353/todo-app:v2
kubectl annotate deployment flask-app kubernetes.io/change-cause="v2: version
heading | health check endpoints"
kubectl rollout status deployment/flask-app
```

Image shows `$ kubectl get pods -w`



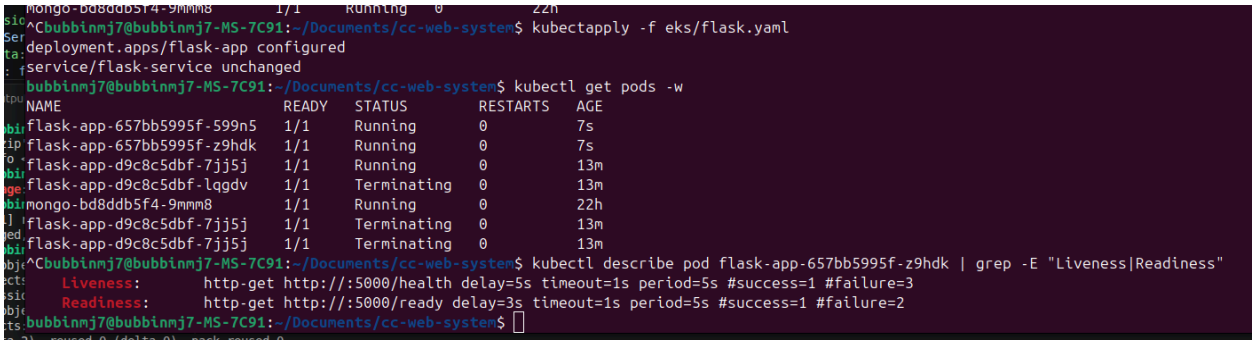


Part 7: Health Monitoring

Add health probes to flask.yaml under containers:

```xml
          ports:
            ---
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 5
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: /ready
              port: 5000
            initialDelaySeconds: 3
            periodSeconds: 5
            failureThreshold: 2
              ----
```

Apply changes to eks/flask and display pod liveliness and readiness:



Now we can test a failure for the first flask pod:

First we can configure the pod to port forward from localhost 8080 to 5000

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl port-forward pod/flask-app-657bb5995f-z9hdk 8080:5000
Forwarding from 127.0.0.1:8080 -> 5000
Forwarding from [::1]:8080 -> 5000
Handling connection for 8080
^Cbubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl describe pod flask-app-657bb5995f-z9hdk | tail -15
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    BestEffort
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason     Age                  From               Message
  ----     ------     ----                 ----               -------
  Normal   Scheduled  4m42s                default-scheduler  Successfully assigned default/flask-app-657bb5995f-z9hdk to ip-192-168-28-98.ec2.internal
  Warning  Unhealthy  72s (x3 over 82s)    kubelet            spec.containers{flask-app}: Liveness probe failed: HTTP probe failed with statuscode: 500
  Normal   Killing    72s                  kubelet            spec.containers{flask-app}: Container flask-app failed liveness probe, will be restarted
  Normal   Pulled     42s (x2 over 4m42s)  kubelet            spec.containers{flask-app}: Container image "mjb9353/todo-app:v2" already present on machine
  Normal   Created    42s (x2 over 4m42s)  kubelet            spec.containers{flask-app}: Created container: flask-app
  Normal   Started    42s (x2 over 4m42s)  kubelet            spec.containers{flask-app}: Started container flask-app
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$
ts: 100% (4/4), 509 bytes | 509.00 KiB/s, done.
```

Then we hit localhost:8080/fail

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ curl localhost:8080/fail
{"status":"liveness will now fail"}
```

With

```
Shell
Kubectl get pods -w
```

We can capture the pod restarting

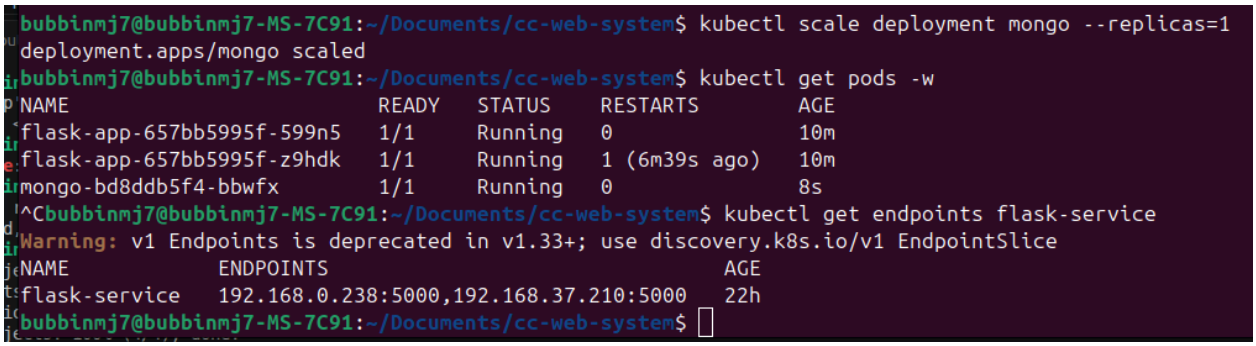
```
inmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods -w
E                       READY   STATUS    RESTARTS      AGE
sk-app-657bb5995f-599n5 1/1     Running   0             2m31s
sk-app-657bb5995f-z9hdk 1/1     Running   0             2m31s
go-bd8ddb5f4-9mmm8      1/1     Running   0             22h
sk-app-657bb5995f-z9hdk 0/1     Running   1 (0s ago)    4m
sk-app-657bb5995f-z9hdk 1/1     Running   1 (6s ago)    4m6s
```

Now we can test the readiness failure for Mongo by simply taking mongo away or “scaling” to 0 replicas.

```
Shell
kubectl scale deployment mongo --replicas=0
```

```
^Cbubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl scale deployment mongo --replicas
deployment.apps/mongo scaled
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods
NAME                            READY   STATUS    RESTARTS        AGE
flask-app-657bb5995f-599n5      1/1     Running   0               9m20s
flask-app-657bb5995f-z9hdk      1/1     Running   1 (5m20s ago)   9m20s
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get endpoints flask-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME            ENDPOINTS   AGE
flask-service               22h
```

Now if we bring mongo back by scaling to 1 replica, we can see that the endpoint returns, and the service is up.



Liveness failures restart the container; readiness failures leave the pod running but remove it from the Service until it recovers.

Part 8: Alerting with Prometheus and Slack (Extra Credit)

I added a monitoring directory but I did not commit this to github because I hardcoded the Slack webhook endpoint into this.
Monitoring/values.yaml

```
XML
grafana:
 enabled: false

# EKS manages these; Prometheus can't scrape them, so don't try
kubeControllerManager:
 enabled: false
kubeScheduler:
 enabled: false
kubeEtcd:
 enabled: false

additionalPrometheusRulesMap:
 todo-rules:
   groups:
    - name: todo-app
      rules:
        - alert: FlaskPodRestarted
          expr:
increase(kube_pod_container_status_restarts_total{namespace="default",
container="flask-app"}[5m]) > 0
          for: 0m
          labels:
            severity: warning
            app: todo
          annotations:
            summary: "Flask pod {{ $labels.pod }} restarted"
            description: "{{ $labels.pod }} was restarted by Kubernetes in the
last 5 minutes (liveness probe failure)."
        - alert: FlaskPodNotReady
          expr: kube_pod_status_ready{namespace="default", condition="false",
pod=~"flask-app-.*"} == 1
          for: 30s
          labels:
            severity: warning
            app: todo
          annotations:
            summary: "Flask pod {{ $labels.pod }} is not ready"
            description: "{{ $labels.pod }} is failing its readiness probe and has
been removed from the Service."
```

```
        - alert: FlaskPodReplaced
          expr: (time() - kube_pod_created{namespace="default",
pod=~"flask-app-.*"}) < 120
          for: 0m
          labels:
            severity: info
            app: todo
          annotations:
            summary: "New Flask pod {{ $labels.pod }} was created"
            description: "A Flask pod was replaced in the last 2 minutes (pod
deleted, crashed, or rolled out)."

alertmanager:
 config:
   global:
     resolve_timeout: 5m
   route:
     receiver: "null"
     group_by: ["alertname", "pod"]
     group_wait: 10s
     group_interval: 1m
     repeat_interval: 12h
     routes:
      - receiver: slack
        matchers:
          - app = "todo"
   receivers:
    - name: "null"
    - name: slack
      slack_configs:
          - api_url: "https://hooks.slack.com/services/<endpoint>"
          channel: "#mjb9353-prometheus-testing"
          send_resolved: true
          title: '{{ if eq .Status "firing" }}🔥{{ else }}✅{{ end }} {{
.CommonLabels.alertname }}'
          text: '{{ range .Alerts }}*{{ .Annotations.summary }}*{{ "\n" }}{{
.Annotations.description }}{{ "\n" }}{{ end }}'
```

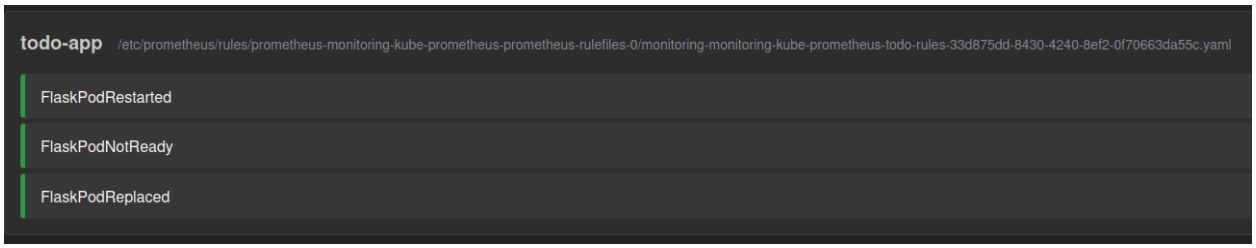
Only alerts labeled `app="todo"` go to Slack, while the stack's built-in alerts (some of which fire constantly on EKS because AWS hides the control plane) go to a `null` receiver. `repeat_interval: 12h` keeps a firing alert from reposting every few hours.

Installation:

```shell
helm repo add prometheus-community
https://prometheus-community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace \
  -f monitoring/values.yaml
```

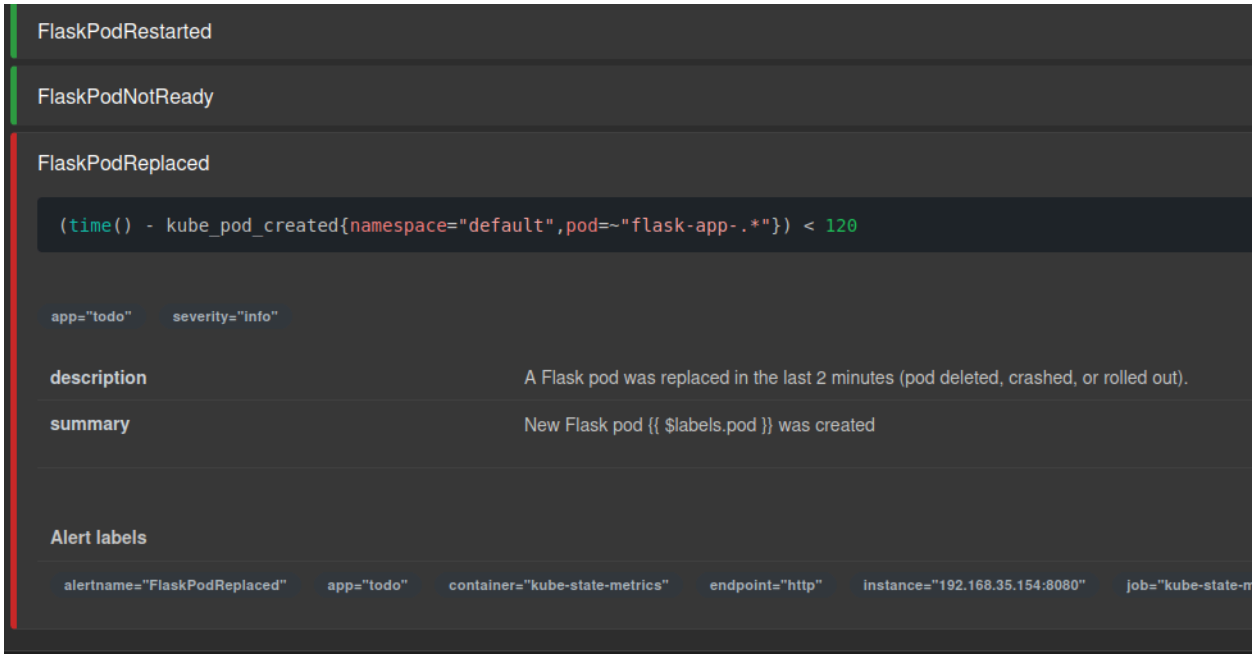
```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods -n monitoring -w
NAME                                                     READY   STATUS    RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0   2/2     Running   0          28s
monitoring-kube-prometheus-operator-5b977778d-h2fxd      1/1     Running   0          32s
monitoring-kube-state-metrics-7b5bd68446-twp95           1/1     Running   0          32s
monitoring-prometheus-node-exporter-6mlf2                1/1     Running   0          32s
monitoring-prometheus-node-exporter-tswjr                1/1     Running   0          32s
prometheus-monitoring-kube-prometheus-prometheus-0       2/2     Running   0          27s
```

These are the three rules from the values.yaml file. The alert for each is inactive.

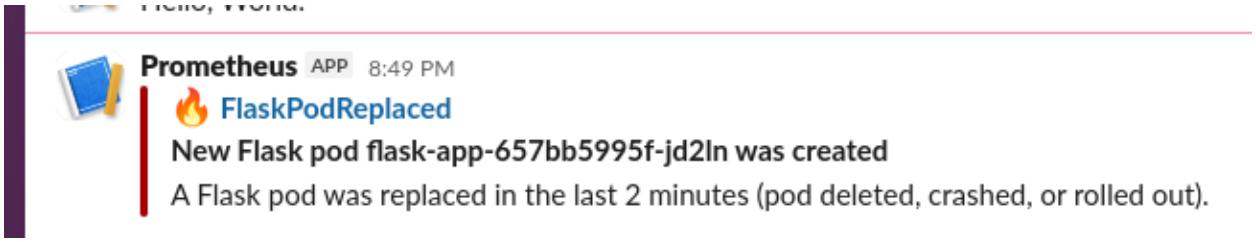


Now lets delete a pod:

```
NAME                         READY   STATUS    RESTARTS   AGE
flask-app-657bb5995f-dbp42   1/1     Running   0          3m5s
flask-app-657bb5995f-wdmc6   1/1     Running   0          8m6s
mongo-bd8ddb5f4-bbwfx        1/1     Running   0          63m
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ ^C
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl delete pod flask-app-657bb5995f-wdmc
pod "flask-app-657bb5995f-wdmc6" deleted from default namespace
```



Now we get an alert in slack:



Liveness check:

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl port-forward pod/flask-app-657bb5995f-dbp42 8080:5000
Forwarding from 127.0.0.1:8080 -> 5000
Forwarding from [::1]:8080 -> 5000
Handling connection for 8080
```

```
^Cbubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ curl localhost:8080/fail
{"status":"liveness will now fail"}
```

> 🔥 **FlaskPodRestarted**
>
> **Flask pod flask-app-657bb5995f-dbp42 restarted**
>
> flask-app-657bb5995f-dbp42 was restarted by Kubernetes in the last 5 minutes (liveness probe failure).

Delete Mongo

Scale down mongo replicas and watch pods respond

Shell

```
kubectl scale deployment mongo --replicas=0
```

```
^Cbubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods -w
NAME                           READY   STATUS    RESTARTS        AGE
flask-app-657bb5995f-dbp42     1/1     Running   1 (2m58s ago)   10m
flask-app-657bb5995f-jd2ln     1/1     Running   0               7m8s
mongo-bd8ddb5f4-bbwfx          1/1     Running   0               71m
mongo-bd8ddb5f4-bbwfx          1/1     Terminating   0               71m
mongo-bd8ddb5f4-bbwfx          1/1     Terminating   0               71m
mongo-bd8ddb5f4-bbwfx          0/1     Completed     0               71m
mongo-bd8ddb5f4-bbwfx          0/1     Completed     0               71m
mongo-bd8ddb5f4-bbwfx          0/1     Completed     0               71m
flask-app-657bb5995f-dbp42     0/1     Running       1 (3m13s ago)   10m
flask-app-657bb5995f-jd2ln     0/1     Running       0               7m23s
```

Bring mongo replica back

```
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl scale deployment mongo --replicas=0
deployment.apps/mongo scaled
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl scale deployment mongo --replicas=1
deployment.apps/mongo scaled
bubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$
```

```
^Cbubbinmj7@bubbinmj7-MS-7C91:~/Documents/cc-web-system$ kubectl get pods -w
NAME                           READY   STATUS              RESTARTS        AGE
flask-app-657bb5995f-dbp42     1/1     Running             1 (2m58s ago)   10m
flask-app-657bb5995f-jd2ln     1/1     Running             0               7m8s
mongo-bd8ddb5f4-bbwfx          1/1     Running             0               71m
mongo-bd8ddb5f4-bbwfx          1/1     Terminating         0               71m
mongo-bd8ddb5f4-bbwfx          1/1     Terminating         0               71m
mongo-bd8ddb5f4-bbwfx          0/1     Completed           0               71m
mongo-bd8ddb5f4-bbwfx          0/1     Completed           0               71m
mongo-bd8ddb5f4-bbwfx          0/1     Completed           0               71m
flask-app-657bb5995f-dbp42     0/1     Running             1 (3m13s ago)   10m
flask-app-657bb5995f-jd2ln     0/1     Running             0               7m23s
mongo-bd8ddb5f4-srggv          0/1     Pending             0               0s
mongo-bd8ddb5f4-srggv          0/1     Pending             0               0s
mongo-bd8ddb5f4-srggv          0/1     ContainerCreating   0               0s
mongo-bd8ddb5f4-srggv          0/1     ContainerCreating   0               6s
mongo-bd8ddb5f4-srggv          1/1     Running             0               6s
flask-app-657bb5995f-dbp42     1/1     Running             1 (4m59s ago)   12m
flask-app-657bb5995f-jd2ln     1/1     Running             0               9m9s
```

Slack messages:

Today

Flask pod flask-app-657bb5995f-jd2ln is not ready

flask-app-657bb5995f-jd2ln is failing its readiness probe and has been removed from the Service.

🔥 **FlaskPodNotReady**

Flask pod flask-app-657bb5995f-dbp42 is not ready

flask-app-657bb5995f-dbp42 is failing its readiness probe and has been removed from the Service.

✅ **FlaskPodNotReady**

Flask pod flask-app-657bb5995f-jd2ln is not ready

flask-app-657bb5995f-jd2ln is failing its readiness probe and has been removed from the Service.

✅ **FlaskPodNotReady**

Flask pod flask-app-657bb5995f-dbp42 is not ready

flask-app-657bb5995f-dbp42 is failing its readiness probe and has been removed from the Service.

✅ **FlaskPodRestarted**

Flask pod flask-app-657bb5995f-dbp42 restarted

flask-app-657bb5995f-dbp42 was restarted by Kubernetes in the last 5 minutes (liveness probe failure).